

do same for use case eda rdd

Here is the complete step-by-step execution guide for **PySpark EDA & RDD Customer Analytics (without Hadoop)**, formatted with system/use-case placeholders enclosed in `<<` `>>`.

TXT

Group A: Environment Setup & Data Ingestion

Terminal 1: Setup Python Virtual Environment & Input Files

1. Verify Python & Java Installations:

Bash



```
python3 --version
pip3 --version
java -version
```

1. Create & Activate PySpark Virtual Environment:

Bash



```
python3 -m venv ~/<<spark-venv>>
source ~/<<spark-venv>>/bin/activate
pip install pyspark==<<3.5.7>>
pyspark --version
```

1. Create Working & Input Directories:

Bash



```
mkdir -p ~/<<telecom-pyspark>>/<<data>>  
cd ~/<<telecom-pyspark>>
```

1. Create Sample Telecom Dataset:

Bash



```
nano <<data/telecom_customers.csv>>
```

Paste Content:

Code snippet



```
<<customer_id>>,<<name>>,<<age>>,<<city>>,<<monthly_bill>>,<<tenure>>,<<plan>>  
C001,Rahul,28,Bengaluru,599,12,Premium  
C002,Priya,35,Mumbai,799,24,Premium  
C003,Amit,42,Delhi,450,36,Basic  
C004,Neha,,Bengaluru,699,18,Premium  
C005,Arun,31,Chennai,,10,Basic  
C006,Kiran,27,Mumbai,999,30,Premium  
C007,Divya,45,Delhi,550,48,Basic  
C008,Ravi,29,Bengaluru,650,15,Premium  
C009,Anita,38,Chennai,500,,Basic  
C010,Suresh,52,Mumbai,1200,60,Premium
```

1. Verify Input CSV File:

Bash



```
cat <<data/telecom_customers.csv>>
```

Group B: PySpark EDA & RDD Script Execution

Terminal 1: Script Execution

1. Create PySpark Analysis Script:

Bash



```
nano <<telecom_eda_rdd.py>>
```

Paste Content:

TXT

Python



```
from pyspark.sql import SparkSession
from pyspark.sql.functions import avg, count, col

# Configure Spark Session for Local Execution (Without Hadoop HDFS)
spark = SparkSession.builder \
    .appName("<<TelecomCustomerAnalytics>>") \
    .master("local[*]") \
    .config("spark.hadoop.fs.defaultFS", "file:///") \
    .getOrCreate()

# Load CSV dataset using PySpark DataFrame
df = spark.read.csv(
    "file:///home/<<ubuntu>>/<<telecom-pyspark>>/<<data/telecom_customers.csv>>",
    header=True,
    inferSchema=True
)

# Data Cleaning: Filter out records with missing age, monthly_bill, or tenure values
df_clean = df.filter(
    col("age").isNotNull() &
    col("monthly_bill").isNotNull() &
    col("tenure").isNotNull()
)

print("\n--- Average Monthly Bill ---")
df_clean.select(
    avg("monthly_bill").alias("average_monthly_bill")
).show()
```

```
print("\n--- Customers By City ---")
df_clean.groupBy("city") \
    .agg(count("*").alias("customer_count")) \
    .orderBy(col("customer_count").desc()) \
    .show()

print("\n--- High Bill Customers (> 700) ---")
df_clean.filter(
    col("monthly_bill") > 700
).show()

print("\n--- Average Bill By Plan ---")
df_clean.groupBy("plan") \
    .agg(avg("monthly_bill").alias("average_bill")) \
    .show()

print("\n--- RDD Demonstration ---")
# Create parallelized RDD
rdd = spark.sparkContext.parallelize([100, 200, 300, 400, 500])

print("Original RDD:")
print(rdd.collect())

# Transformation: Map (Double values)
rdd_doubled = rdd.map(lambda x: x * 2)
print("RDD After map (doubled):")
print(rdd_doubled.collect())

# Transformation: Filter (> 250)
rdd_filtered = rdd.filter(lambda x: x > 250)
print("RDD After filter (> 250):")
print(rdd_filtered.collect())

# Action: Count
print("RDD Count:")
print(rdd_filtered.count())

print("\n--- DataFrame Caching Demonstration ---")
df_clean.cache()

print("First Count (Populates Cache):")
print(df_clean.count())

print("Second Action (Reads from Cache):")
print(df_clean.select(avg("monthly_bill")).collect())

# Stop Spark Session
```

```
spark.stop()
print("\nSpark Application Completed Successfully.")
```

1. Execute PySpark Script:

Bash



```
spark-submit <<telecom_eda_rdd.py>>
```

Group C: Interactive PySpark Shell Option (Optional)

Terminal 1: PySpark Shell

1. Launch PySpark Interactive Shell:

Bash



```
pyspark
```

1. Run RDD & EDA Operations Interactively:

Python



```
# Create test RDD
rdd = spark.sparkContext.parallelize([100, 200, 300, 400, 500])

# Perform Map and Filter
rdd_doubled = rdd.map(lambda x: x * 2)
print(rdd_doubled.collect())

rdd_filtered = rdd.filter(lambda x: x > 250)
print(rdd_filtered.collect())
print(rdd_filtered.count())

# Exit Shell
exit()
```

